

Modulo complementario: UTE Complementos Academicos

Objetivo

Se desarrollo el modulo `odoo_ute_complementos` como complemento del modulo existente `odoo_ute`. No se reemplaza el modulo base: se aprovechan sus modelos `ou.teacher` y `ou.signature` para crear una extension coherente de gestion academica.

El modulo base `odoo_ute` ya administra docentes y materias. El complemento agrega cinco modelos nuevos relacionados con esos datos:

1. `ou.academic.plan`: planificacion academica del docente.
2. `ou.classroom`: aulas y espacios academicos.
3. `ou.class.schedule`: horarios de clase.
4. `ou.attendance`: asistencia docente.
5. `ou.teacher.evaluation`: evaluacion docente.

Tambien se hereda el modelo `ou.teacher` para mostrar informacion complementaria dentro de la ficha del docente.

Estructura del addon

Ruta del modulo:

`odoo/custom-addons/odoo_ute_complementos`

Estructura:

```
odoo_ute_complementos/
├── __init__.py
├── __manifest__.py
├── models/
│   ├── __init__.py
│   ├── academic_plan.py
│   ├── attendance.py
│   ├── class_schedule.py
│   ├── classroom.py
│   ├── teacher_evaluation.py
│   └── teacher_extension.py
├── security/
│   └── ir.model.access.csv
```

```

├── record_rules.xml
├── security.xml
├── views/
│   ├── academic_plan_views.xml
│   ├── attendance_views.xml
│   ├── class_schedule_views.xml
│   ├── classroom_views.xml
│   ├── menu_views.xml
│   ├── teacher_evaluation_views.xml
│   └── teacher_extension_views.xml

```

El archivo `__manifest__.py` declara la dependencia con `odoo_ute`, por eso Odoo instala primero el modulo base y luego el complemento.

Relacion con el modulo existente

El modulo base tiene:

- `ou.teacher`: docentes de la UTE.
- `ou.signature`: materias.

El complemento usa esos modelos mediante campos `Many2one`, `One2many` y una herencia:

- `ou.academic.plan.teacher_id` relaciona planes con docentes.
- `ou.academic.plan.signature_id` relaciona planes con materias.
- `ou.class.schedule.teacher_id` relaciona horarios con docentes.
- `ou.class.schedule.signature_id` relaciona horarios con materias.
- `ou.attendance.teacher_id` relaciona asistencias con docentes.
- `ou.teacher.evaluation.teacher_id` relaciona evaluaciones con docentes.
- `teacher_extension.py` usa `_inherit = "ou.teacher"` para agregar campos calculados y listas relacionadas dentro del docente.

Modelo 1: `ou.academic.plan`

Representa la planificacion academica de un docente para una materia y un periodo.

Campos:

- `name`: Char, nombre del plan.
- `teacher_id`: `Many2one` a `ou.teacher`.
- `signature_id`: `Many2one` a `ou.signature`.
- `period`: `Selection`, periodo academico.

- `start_date`: Date, fecha de inicio.
- `end_date`: Date, fecha de fin.
- `weekly_hours`: Integer, horas semanales.
- `total_weeks`: Integer, numero de semanas.
- `total_hours`: Integer calculado.
- `state`: Selection, borrador, aprobado o cerrado.
- `active`: Boolean, estado activo.
- `objective`: Text, objetivo del plan.

Decoradores:

- `@api.depends`: calcula `total_hours` multiplicando horas semanales por semanas.
- `@api.onChange`: al elegir docente, toma como materia sugerida la materia principal del docente.
- `@api.constrains`: valida horas, semanas y rango de fechas.

Vistas y menu:

- Vista `list`.
- Vista `form`.
- Accion `action_ou_academic_plan`.
- Menu `Planes academicos`.

Modelo 2: `ou.classroom`

Representa aulas fisicas o virtuales.

Campos:

- `name`: Char, nombre del aula.
- `code`: Char, codigo unico.
- `building`: Char, edificio.
- `floor`: Integer, piso.
- `capacity`: Integer, capacidad.
- `room_type`: Selection, aula normal, laboratorio, auditorio o virtual.
- `has_projector`: Boolean, indica si tiene proyector.
- `has_computers`: Boolean, indica si tiene computadores.
- `active`: Boolean, aula activa.
- `display_label`: Char calculado.

- notes: Text, observaciones.

Decoradores:

- `@api.depends`: arma la etiqueta del aula con codigo, nombre y edificio.
- `@api.onchange`: si el aula es laboratorio activa computadores y proyector; si es virtual aumenta la capacidad.
- `@api.constrains`: valida capacidad positiva y piso no negativo.

Vistas y menu:

- Vista list.
- Vista form.
- Accion `action_ou_classroom`.
- Menu Aulas.

Modelo 3: `ou.class.schedule`

Representa horarios de clase conectados con docente, materia, aula y plan academico.

Campos:

- name: Char calculado.
- teacher_id: Many2one a `ou.teacher`.
- signature_id: Many2one a `ou.signature`.
- classroom_id: Many2one a `ou.classroom`.
- academic_plan_id: Many2one a `ou.academic.plan`.
- day_of_week: Selection, dia de clase.
- start_hour: Float, hora inicio.
- end_hour: Float, hora fin.
- duration: Float calculado.
- modality: Selection, presencial, virtual o hibrida.
- active: Boolean, horario activo.
- observations: Text, observaciones.

Decoradores:

- `@api.depends`: calcula la referencia del horario y la duracion.
- `@api.onchange`: al elegir un plan academico, carga docente y materia automaticamente.

- `@api.constrains`: valida que las horas esten entre 0 y 24 y que la hora final sea mayor.

Vistas y menu:

- Vista `list`.
- Vista `form`.
- Accion `action_ou_class_schedule`.
- Menu Horarios.

Modelo 4: `ou.attendance`

Representa la asistencia del docente a una clase.

Campos:

- `name`: Char calculado.
- `teacher_id`: Many2one a `ou.teacher`.
- `schedule_id`: Many2one a `ou.class.schedule`.
- `signature_id`: Many2one a `ou.signature`.
- `attendance_date`: Date, fecha.
- `check_in`: Float, hora entrada.
- `check_out`: Float, hora salida.
- `worked_hours`: Float calculado.
- `status`: Selection, presente, atrasado, ausente o justificado.
- `justified`: Boolean, indica justificacion.
- `notes`: Text, notas.

Decoradores:

- `@api.depends`: calcula la referencia y las horas trabajadas.
- `@api.onchange`: al elegir horario, carga docente, materia y horas.
- `@api.constrains`: valida que las horas sean correctas.

Vistas y menu:

- Vista `list`.
- Vista `form`.
- Accion `action_ou_attendance`.
- Menu Asistencias.

Modelo 5: ou.teacher.evaluation

Representa evaluaciones academicas del docente.

Campos:

- name: Char calculado.
- teacher_id: Many2one a ou.teacher.
- signature_id: Many2one a ou.signature.
- evaluation_date: Date, fecha de evaluacion.
- evaluator: Char, evaluador.
- methodology_score: Float, nota de metodologia.
- punctuality_score: Float, nota de puntualidad.
- content_score: Float, nota de dominio de contenidos.
- average_score: Float calculado.
- result: Selection calculado.
- approved: Boolean calculado.
- comments: Text, comentarios.

Decoradores:

- @api.depends: calcula referencia, promedio, resultado y aprobado.
- @api.onchange: al elegir docente, sugiere su materia principal.
- @api.constrains: valida que las calificaciones esten entre 0 y 10.

Vistas y menu:

- Vista list.
- Vista form.
- Accion action_ou_teacher_evaluation.
- Menu Evaluaciones.

Seguridad

Se crearon dos grupos complementarios dentro del privilegio UTE:

- Usuario Complementos: hereda el acceso de odoo_ute.group_ute_user.
- Administrador Complementos: hereda el acceso de Usuario Complementos y de odoo_ute.group_ute_manager.

Permisos por modelo:

- Usuario Complementos: solo lectura.
- Administrador Complementos: lectura, escritura, creacion y eliminacion.

Esto esta definido en:

```
security/security.xml
security/ir.model.access.csv
security/record_rules.xml
```

Ademas, los menus del complemento estan protegidos con los grupos de seguridad, por lo que solo aparecen a usuarios autorizados.

Menus y acciones

El menu principal del modulo base es UTE. Los menus del complemento se agregaron directamente al mismo nivel que Docentes y Materia, para mantener la estructura original del modulo base:

```
UTE
├── Docentes
├── Materia
├── Planes academicos
├── Aulas
├── Horarios
├── Asistencias
└── Evaluaciones
```

Cada opcion tiene su accion `ir.actions.act_window` y abre su modelo con vista `list`, `form`.

Ajuste de configuracion

Se corrigio `odoo.conf` para que el `addons_path` apunte a:

```
odoo/custom-addons
```

Esta es la carpeta correcta donde esta el modulo base `odoo_ute` y el nuevo complemento `odoo_ute_complementos`.

Verificacion realizada

Se verifico:

- Sintaxis Python con `python3 -m compileall`.
- XML bien formado con `xmllint --noout`.
- Eliminacion de los modulos incorrectos `jj_academia_*`.

- Existencia de vistas, acciones, menus, seguridad y reglas de acceso.

Anexo final: captura de GitHub

Despues de subir los cambios al repositorio:

https://github.com/czambrano1997/ProgII_UTE202601

se debe adjuntar al final del PDF una captura donde se vea el ultimo commit de la rama del estudiante.